

WEEK 1 · PYTHON PROGRAMMING

# Python Overview & Development Environment

Variables, data types, input/output and type conversion

---

Theory 2 h · Practice 2 h  
Instructor: MinGi Kyung

# Today's Agenda

---

## Part 1 • Theory (2 h)

- Python overview: what it is and why it matters
- Development environment: VS Code, virtual environments (venv), Google Colab
- Variables and data types
- Input and output with `input()` and `print()`
- Type conversion (casting)

## Part 2 • Practice (2 h)

- Lab 1: set up your development environment
- Lab 2: Hello, World!
- Lab 3: input and calculation program
- Lab 4: BMI calculator
- Submit your three `.py` files at the end of class

*Course plan: weeks 1–8 cover the Python core, weeks 9–16 are a team term project.*



## PART 1 · THEORY

# Python Overview & Development Environment

- What is Python and how does it run?
- Local setup with VS Code and venv, or Google Colab in the browser
- Variables, data types, print() / input(), type conversion

# What Is Python?

---

- Created by **Guido van Rossum**, first released in 1991
- High-level, general-purpose language with a clear, readable syntax
- **Interpreted**: run code line by line, no separate compile step
- **Dynamically typed**: no need to declare variable types
- Huge standard library plus 500,000+ packages on PyPI
- Python 3 is the current version (Python 2 retired in 2020)

```
# Python reads almost like English
name = "Python"
print("Hello,", name)

# Java needs a lot more to say the same
# public class Main {
#     public static void main(String[] a) {
#         System.out.println("Hello, Java");
#     }
# }
```

# Why Learn Python?

---

## Data & AI

- pandas, NumPy, matplotlib
- scikit-learn, PyTorch, TensorFlow
- The default language of data science

## Web & Automation

- Django, Flask, FastAPI
- Scripts, bots, web scraping
- Glue code between systems

## Everywhere Else

- Education and scientific computing
- Games, DevOps, cloud tooling
- Consistently #1 in language rankings

- Beginner-friendly syntax lets you focus on **problem solving** instead of language details
- One language for the whole course: core syntax (weeks 1–8) and the team project (weeks 9–16)

# How Python Runs Your Code

---

- Source code (`.py`) is compiled to **bytecode**, then executed by the Python Virtual Machine
- **Interactive mode (REPL)**: type `python3`, enter statements one at a time, see results instantly
- **Script mode**: save a file and run it with `python3 file.py`
- **Notebook mode** (Colab, Jupyter): cells combine both styles
- Check your version with `python3 --version`; on Windows the command is usually `python` or `py`

```
$ python3 --version
Python 3.12.4
```

```
$ python3                                # interactive mode
>>> 2 + 3
5
>>> print("hi")
hi
>>> exit()
```

```
$ python3 hello.py                       # script mode
Hello, World!
```

# Development Environment: Three Options

---

## Local: Python + VS Code

- Full control, works offline
- Best for real projects and assignments
- Our main setup in class

## Virtual Environment (venv)

- One isolated set of packages per project
- Avoids version conflicts
- Standard practice for every project

## Google Colab

- Runs in the browser, nothing to install
- Free GPU for heavier work
- Great for quick experiments and data work

- Recommended: **VS Code + venv** for assignments, **Colab** as a backup or when away from your laptop
- Everything you learn works the same in all three: it is the same Python

# Setting Up VS Code

---

1. Install **Python 3.x** from python.org (Windows: tick “Add python.exe to PATH”)
2. Install **VS Code** from code.visualstudio.com
3. Add the **Python** extension by Microsoft from the Extensions view
4. Open a folder: File → Open Folder... (e.g. `python-week1`)
5. Select the interpreter: Ctrl/Cmd + Shift + P → “Python: Select Interpreter”
6. Create `hello.py`, then press the ► Run button or use the terminal

```
# Open the built-in terminal:
#   Ctrl + `   (Windows / Linux)
#   Cmd + `    (macOS)

$ python3 --version
Python 3.12.4

$ python3 hello.py
Hello, World!
```

*Tip: the selected interpreter is shown in the bottom-right status bar.*

# Virtual Environments with venv

---

- Each project gets its **own** Python packages, isolated from the system
- Prevents version conflicts (two projects needing different pandas versions)
- Built into Python: nothing extra to install
- Always **activate** before `pip install`; VS Code detects `.venv` automatically
- Add `.venv/` to `.gitignore` and share a `requirements.txt` instead

```
# 1. Create (inside your project folder)
$ python3 -m venv .venv

# 2. Activate
$ source .venv/bin/activate # macOS/Linux
> .venv\Scripts\activate    # Windows

# 3. Install packages
(.venv) $ pip install requests

# 4. Leave the environment
(.venv) $ deactivate
```

# Google Colab

---

- Jupyter notebooks hosted by Google:  
**colab.research.google.com**
- Nothing to install: only a browser and a Google account
- Code cells and text (Markdown) cells; run a cell with **Shift + Enter**
- Popular libraries are pre-installed; free GPU/TPU for heavier work
- Notebooks ([.ipynb](#)) are saved to Google Drive and shared like a Google Doc
- Watch out: the runtime resets after idle time, so save files to Drive

```
# In a Colab code cell
print("Hello from Colab!")

# Shell commands start with !
!python --version
!pip install some-package

# Variables persist between cells
x = 10
```

# Variables

---

- A variable is a **name** that refers to a value stored in memory
- Created by assignment: `name = value` (no declaration keyword)
- Reassignment can change the value and even the type
- Names: letters, digits and underscore; cannot start with a digit; case-sensitive
- Keywords such as `if`, `for`, `class` cannot be used as names
- Convention: `snake_case` for variables, `UPPER_CASE` for constants

```
age = 20
height = 175.5
name = "Kim"
is_student = True

age = age + 1          # 21
age = "twenty-one"    # now a string!

# Multiple assignment
x, y = 10, 20
MAX_SCORE = 100       # a constant
```

# Basic Data Types

---

| Type                  | Examples                     | Description                              |
|-----------------------|------------------------------|------------------------------------------|
| <code>int</code>      | <code>42, -7, 0</code>       | Whole numbers, unlimited size            |
| <code>float</code>    | <code>3.14, 2.0, 1e-3</code> | Decimal numbers (64-bit floating point)  |
| <code>str</code>      | <code>"hello", 'a'</code>    | Text; single or double quotes            |
| <code>bool</code>     | <code>True, False</code>     | Logical values (note the capital letter) |
| <code>NoneType</code> | <code>None</code>            | The absence of a value                   |

- Check a value's type with `type(x)`, for example `type(3.14)` gives `<class 'float'>`
- The type decides which operations are allowed: `"3" + 3` is an error, `"3" + "3"` is `"33"`

# Output with print()

---

- `print()` writes values to the screen, separated by a space
- `sep=` changes the separator, `end=` changes the line ending (default: newline)
- **f-strings** (`f"..."`) embed values directly in text: the modern way
- Format specifiers: `{x:.2f}` gives 2 decimals, `{n:,}` adds thousands separators

```
print("Hello", "World")      # Hello World
print("A", "B", sep="-")    # A-B
print("same line", end=" ")
print("continues")

name, score = "Kim", 95.5
print(f"{name} scored {score}")
# Kim scored 95.5

print(f"{score:.1f} pts")    # 95.5 pts
print(f"{1234567:,}")        # 1,234,567
```

# Input with input()

---

- `input(prompt)` shows the prompt, waits for the user, and returns what they typed
- The result is **always a string**, even when the user types a number
- Convert with `int()` or `float()` before doing arithmetic
- `"3" + "4"` gives `"34"` (concatenation), not 7
- Typical pattern: `n = int(input("Number: "))`

```
name = input("Your name: ")
print(f"Hi, {name}!")

age = input("Your age: ")
print(type(age))          # <class 'str'>
print(age + 1)            # TypeError!

age = int(input("Your age: "))
print(age + 1)            # works now
```

# Type Conversion (Casting)

---

| Function             | Input → Output                                    | Notes                                                 |
|----------------------|---------------------------------------------------|-------------------------------------------------------|
| <code>int()</code>   | <code>"42" → 42</code> , <code>3.9 → 3</code>     | Truncates floats; <code>int("3.5")</code> is an error |
| <code>float()</code> | <code>"3.5" → 3.5</code> , <code>7 → 7.0</code>   | Accepts integers and numeric strings                  |
| <code>str()</code>   | <code>42 → "42"</code> , <code>3.5 → "3.5"</code> | Needed before joining a number with text              |
| <code>bool()</code>  | <code>0, "", None → False</code>                  | Everything else becomes <code>True</code>             |

- **Implicit** conversion happens in arithmetic: `int + float` gives `float`
- **Explicit** conversion is your job between strings and numbers, e.g. `int(float("3.5"))`



## PART 2 · PRACTICE

# Hands-on Labs

- Lab 1: set up your environment
- Lab 2: Hello, World!
- Lab 3: input and calculation program
- Lab 4: BMI calculator

# Lab 1 · Set Up Your Environment

---

1. Install Python 3.x and confirm with `python3 --version`
2. Install VS Code and the Python extension
3. Create a folder `python-week1` and open it in VS Code
4. Create a virtual environment `.venv` and activate it
5. Select the `.venv` interpreter in VS Code (bottom-right status bar)
6. Open Google Colab and create a new notebook as a backup

```
$ mkdir python-week1
$ cd python-week1
$ python3 -m venv .venv
$ source .venv/bin/activate
(.venv) $ python --version
Python 3.12.4
(.venv) $ code .

# Windows: .venv\Scripts\activate
```

*Done when the terminal prompt starts with `(.venv)` and VS Code shows the `.venv` interpreter.*

# Lab 2 · Hello, World!

---

- Create `hello.py` in VS Code and type the program on the right
- Run it with the ▶ button or `python hello.py` in the terminal
- Run the same code in a Colab cell with Shift + Enter
- Extend it: print your name, your major and today's date on separate lines
- Experiment: remove a quotation mark and **read the error message**

```
# hello.py
print("Hello, World!")
print("Hello, Python!")

# Your turn
print("Name:", "Kim Minsu")
print("Major:", "Computer Science")
print("Week", 1, "of", 16)
```

# Lab 3 · Input & Calculation

---

- Read two numbers from the user with `input()`
- Print their **sum, difference, product and quotient**
- Format the quotient to 2 decimal places with an f-string
- Challenge: also print integer division `//`, remainder `%` and power `**`
- Challenge: what happens when the second number is 0? (We will fix this in week 5)

```
# calc.py
a = float(input("First number: "))
b = float(input("Second number: "))

print(f"{a} + {b} = {a + b}")
print(f"{a} - {b} = {a - b}")
print(f"{a} * {b} = {a * b}")
print(f"{a} / {b} = {a / b:.2f}")
```

# Lab 4 · BMI Calculator

---

- BMI = weight (kg) ÷ height (m)<sup>2</sup>
- Read height in **cm** and weight in **kg**, then convert cm to m
- Print the BMI with 1 decimal place
- Challenge (preview of week 2): print a category
  - Under 18.5: Underweight
  - 18.5 – 22.9: Normal
  - 23 – 24.9: Overweight
  - 25 and above: Obese (Asia-Pacific cut-offs)

```
# bmi.py
height_cm = float(input("Height (cm): "))
weight = float(input("Weight (kg): "))

height_m = height_cm / 100
bmi = weight / (height_m ** 2)

print(f"Height: {height_m:.2f} m")
print(f"Weight: {weight:.1f} kg")
print(f"Your BMI is {bmi:.1f}")
```

*Test with 175 cm and 70 kg: expected BMI 22.9.*

# Summary & Next Week

---

## Today

- Python: interpreted, dynamically typed, readable
- Tools: VS Code + venv locally, Colab in the browser
- Variables and the core types `int`, `float`, `str`, `bool`
- `print()` with f-strings; `input()` always returns `str`
- Casting with `int()`, `float()`, `str()`, `bool()`

## Week 2

- Operators: arithmetic, comparison, logical
- String processing: indexing, slicing, methods
- Conditionals: `if` / `elif` / `else`
- Labs: string manipulation, grade classifier, rock-paper-scissors, leap year

*Assignment: submit `hello.py`, `calc.py` and `bmi.py` to the LMS before next class.*



# Questions?

Python Programming · Week 1

MinGi.KYUNG@gmail.com

Office hours: see the syllabus